M0: Project setup, toolchain and verification machinery

itchbook walkthrough, milestone 0. What was built before any market data was parsed, why each piece is there, and what you should be able to say about it in an interview.

What the project is, and why the order of the milestones matters

itchbook reads NASDAQ TotalView-ITCH 5.0 files, which are the exchange's own record of every order event in a trading day, and rebuilds the full order-by-order book from them. Every order resting at every price, in queue order, is known at every nanosecond of the day. That reconstruction is called an L3 book. Most retail and academic data is L2, which only tells you the total size at each price level. Once the L3 book is correct, it becomes ground truth for two research questions. The first is how wrong the usual L2 models of queue position are, which we can now measure directly because we know the true queue. The second is whether deep learning models of the order book actually beat a simple order-flow-imbalance regression once trading costs are charged.

The whole project is a stack in which each layer is the measuring instrument for the next one. The research is only as trustworthy as the book. The book is only as trustworthy as the parser. The parser is only as trustworthy as the tests that check it. So the first milestone builds no finance at all. It builds the machinery that lets every later claim be checked against something that cannot fake agreement: an independent implementation, an exhaustive enumeration, a published number, or a sanitizer that watches every memory access. When an interviewer asks how you know a number is right, the honest answer should always name one of these checks.

The build: CMake, targets and pinned dependencies

C++ has no standard build tool. CMake is the de facto choice: a meta-build system that reads a description of the project and generates build files for a lower-level tool, here Ninja. Ninja does nothing but run compile commands in the right order as fast as possible. The description is organised around targets. A target is a library or executable together with its sources, include paths, compile flags and dependencies, and properties flow from a library to whatever links it. `itchbook_core` is the one library. Its public include directory and its single external dependency, zlib, propagate automatically to the `itch-stats` tool, the tests, the benchmarks and the fuzzer. Keeping the core's dependencies down to zlib is deliberate. Anything you can't explain line by line in an interview is a liability in the engine itself, while test tooling can be as heavy as it needs to be.

The language standard is C++20 with compiler extensions turned off, so the code compiles as the standard defines it and not as one compiler's dialect. The C++20 features used are concrete rather than decorative:

- `std::span` passes byte ranges without copying them;
- `std::endian` lets the byte-order code check at compile time whether the host is little-endian;
- template lambdas build a lookup table of message sizes at compile time;
- `[[likely]]` and `[[unlikely]]` attributes hint which branches the hot path takes.

Test dependencies are fetched by CMake's FetchContent at exact revisions:

- GoogleTest v1.15.2;
- Google Benchmark v1.9.4;
- RapidCheck at a fixed commit hash, because RapidCheck has never published tagged releases.

Pinning matters for reproducibility. A build that pulls "latest" can change behaviour under you between two runs of a benchmark, and then you can't say which change moved the number. CMake presets (`CMakePresets.json`) name the configurations so they're one command each:

- `release` is the portable optimized build CI uses;
- `native` is tuned for the machine it builds on, for benchmarks only;
- `asan` adds the sanitizers;
- `fuzz` builds the libFuzzer target;
- `dev` is a plain debug build.

Compiler warnings as a first line of defence

Every project target compiles with a strict warning set, and warnings are errors in every preset. The most valuable warnings for this code are `-Wconversion` and `-Wsign-conversion`. They flag every implicit narrowing or sign change, such as assigning a 64-bit value to a 32-bit variable or mixing signed and unsigned arithmetic. A parser that decodes 2-, 4-, 6- and 8-byte unsigned integers and then does arithmetic on share counts and prices is exactly where a silent truncation hides. The habit carries over from holdem-cfr, where a 32-bit float accumulator silently stopped absorbing updates past 2^24. Here every price is the exchange's own fixed-point integer, and every counter is 64 bits.

One warning was deliberately removed: `-Wnull-dereference`. On GCC 13 at -O3 it produces false positives inside the standard library's own `std::vector` code, after inlining. Keeping it would mean either sprinkling suppressions or disabling

optimization, so it went. This is the kind of decision you should be ready to defend: a warning that fires on correct code trains people to ignore warnings, which is worse than not having it. Null dereferences are still caught at run time by AddressSanitizer, described next.

Sanitizers: making undefined behaviour loud

C++ has undefined behaviour. Reading past the end of a buffer, using memory after it is freed, overflowing a signed integer, or loading a 4-byte integer from an address that isn't a multiple of four can all appear to work in testing. Then they corrupt results or crash later, somewhere unrelated, because the compiler is allowed to assume these things never happen and optimises accordingly. Sanitizers are compiler instrumentation that checks for these events while the program runs.

AddressSanitizer (ASan) keeps "shadow memory": for every 8 bytes of program memory, one shadow byte records how many of them are addressable. Every heap and stack allocation is surrounded by poisoned "redzones". Every load and store is preceded by a shadow check, so reading a single byte past a buffer stops the program with the exact line and allocation. UndefinedBehaviorSanitizer (UBSan) inserts checks for arithmetic and type-level undefined behaviour, including misaligned pointer access. Both are configured to be fatal (`-fno-sanitize-recover=all`), so a single report fails the test run instead of scrolling past. The cost is roughly a 2–3× slowdown, which is why they run in a separate CI job and not in the benchmark build.

This parser in particular needs them. ITCH messages are packed back to back with no alignment, so a 4-byte price can start at any byte. The tempting shortcut is to cast the byte pointer to a `uint32_t*` and dereference it. That is undefined behaviour, and UBSan's alignment check catches it. The engine instead copies the bytes with `memcpy` into a local integer and byte-swaps it. Compilers recognise this pattern and emit a single unaligned load instruction (x86 and ARM64 both support those), so the safe version costs nothing. The full test suite passes under ASan and UBSan with zero reports.

Three kinds of testing, and why each one exists

Example-based tests, written with GoogleTest, check specific inputs against specific expected outputs. For the parser, every one of the 23 message types has a test that encodes a message field by field and checks every accessor. The encoder appends fields one after another, the way a `struct` format string does, while the decoder reads each field at an explicit byte offset. The two descriptions of the layout were written separately, so an offset typo in either one shows up as a mismatch. The test values deliberately set the top bit of each field width, so a sign-extension or truncation bug can't pass by accident.

Property-based tests, written with RapidCheck, state something that must hold for every input and let the library generate hundreds of random inputs to try to break it. The central property is: "for any sequence of valid messages, cut into chunks of any size and fed through a read buffer of any size, the parser delivers exactly those messages, byte for byte". When a property fails, RapidCheck shrinks the failing input to a minimal counterexample, such as the shortest message list and the smallest chunk size that still fail. That makes the bug easy to diagnose. Example-based tests only check the cases you thought of. Property tests probe the cases you didn't, and buffer-boundary bugs are exactly the kind people don't think of.

Fuzzing, done with libFuzzer, goes a step further with no oracle at all. The fuzzer feeds the parser arbitrary bytes and watches which branches of the code each input reaches, using coverage instrumentation the compiler inserts. Inputs that reach new branches are kept and mutated further, so over time it learns to produce valid length prefixes, then valid type bytes, then messages split across buffer reads. Combined with ASan and UBSan, any memory error it provokes becomes a crash with a reproducer file. The target also checks two accounting invariants on every input. First, the bytes consumed plus the incomplete tail equal the input size. Second, every frame is classified exactly once: delivered, unknown type, wrong length, or end-of-session marker. A 60-second local run executed about 2.5 million inputs with no failure. CI fuzzes for two minutes on every push. Apple clang ships without the libFuzzer runtime, so fuzzing runs on Linux (CI or the desktop under WSL2), or on the Mac with Homebrew LLVM.

Benchmarks: Google Benchmark, and what the first numbers say

Google Benchmark repeats a function until the timing is statistically stable. It chooses the iteration count itself and reports time per iteration together with user counters such as items and bytes per second. `benchmark::DoNotOptimize` stops the compiler from deleting work whose result is never used, which would otherwise make the benchmark measure nothing. The benchmark input is an in-memory stream of two million messages. Its mix of message types follows the published counts for the 01302020 trading day: 44% adds, 43% deletes, 9% replaces, and so on.

The first numbers come from a cloud Xeon with the portable release build, so they are indicative only; the real numbers come in M3. They are:

- about 10 ns per message (99 million messages per second) to frame and dispatch;
- about 11 ns when the fields a book needs are decoded;
- about 220 MB/s for zlib to decompress the synthetic stream.

Two caveats matter.

The synthetic message bodies are random bytes. Random bytes barely compress, so the gzip figure is pessimistic: real ITCH compresses roughly 2.3:1 and decompresses faster.

The more important conclusion is structural. A full day is about 13 GB uncompressed and about 400 million messages, so parsing takes roughly 4 seconds while single-threaded zlib takes roughly 30 seconds or more. End to end, decompression will dominate. That is why the performance milestone has to separate the book's cost from the cost of reading input. CppTrader's published figures are measured on an uncompressed file piped through standard input, so a fair comparison runs both engines on the same uncompressed bytes. Faster decompression (zlib-ng, libdeflate, or decompressing on a second core) is a separate, honest optimisation, and it has to be reported separately.

Static analysis and formatting

`clang-format` enforces a single code style mechanically, so style never comes up in review. `clang-tidy` is a static analyser built on the compiler's own parser. It is configured for correctness and performance check families: bug-prone patterns, the Clang static analyser, modernisation, and performance pitfalls. It skips style rules, which are clang-format's job. Two families were removed because they produced noise instead of signal. `cert-err33-c` demands that the return value of every `fprintf` be checked. `concurrency-mt-unsafe` flags `strerror` in a single-threaded tool. CI fails on any remaining finding.

The Python side

Analysis and the independent reference parser use Python managed by `uv`, a fast package and environment manager. `pyproject.toml` declares the environment and `uv.lock` pins every version exactly, so `uv run pytest` gives the same environment on the Mac, the desktop and CI. The reference parser uses only the standard library. Fewer dependencies means fewer things that could make the two implementations agree for the wrong reason.

Continuous integration

GitHub Actions runs four groups of jobs on Ubuntu 24.04 for every push.

The first group builds and tests with GCC 13 and with Clang 18, plus a Clang build under ASan and UBSan. Two compilers matter: they warn about different things and they exploit undefined behaviour differently, so code that only works by accident on one tends to break on the other.

The second job runs the Python lint, the reference parser's tests, and the mutation tests described in the M1 walkthrough.

The third runs clang-format and clang-tidy.

The fourth builds the fuzzer and fuzzes for two minutes.

The M0 gate is "CI green, sanitizers clean". Both hold locally. CI goes green the first time the repository is pushed to GitHub, and the Mac isn't used for CI at all. macOS runners cost ten times as many free-tier minutes as Linux on private repositories, and the Mac build is exercised every time you build locally.

The data, and two surprises from checking it

NASDAQ publishes free sample days at `emi.nasdaq.com`. Checking the server directly from the Mac changed two assumptions in the original plan.

First, the 30 January 2017 file is gone. That is the day CppTrader's published benchmark (120 ns per message for its optimised book) was measured on. Benchmarking CppTrader on its own published file is therefore impossible. The plan was always to build CppTrader and run both engines on the same file on the same machine, and that is still possible on any day, so the head-to-head comparison is unaffected. The published 120 ns becomes context, not the target. We use 30 January 2020 as the gate and benchmark day, because a third party has published per-type message counts for exactly that file: 423,285,709 messages in total. That gives the parser an extra external check.

Second, the `.md5sum` checksum files are listed but return HTTP 404. The server is Microsoft IIS, which by default refuses to serve file extensions it has no MIME type for, and `.md5sum` and `.done` are not registered. Integrity therefore rests on two other checks, which together are stronger than an MD5 sum. Every gzip member ends with a CRC32 of its uncompressed contents and its length, and `gzip -t` verifies them, so a corrupted or truncated download is detected. Then a SHA-256 of the verified file is recorded in `data/manifest.tsv`, which pins its identity, so any later re-download must produce exactly the same bytes. The download script resumes interrupted transfers and runs both checks. The first bytes of the files were inspected from the Mac: the 2019–2020 files and the newer `S*-v50.txt.gz` files are binary ITCH 5.0 with 2-byte length framing, despite the `.txt` in the name.

The data lives in `~/CODE/itch-data`, outside every repository. The files are 3.5–18 GB compressed and 10–40 GB uncompressed, more than the 8 GB of RAM in the development Mac. That is the reason the engine streams: it never holds more than a fixed buffer of about one megabyte of input, whatever the file size.

One planning consequence for M3: CppTrader's optimised benchmark keeps a flat array of 300 million order slots indexed by order reference number. That needs several gigabytes of RAM before any data is read, and the 2020 file has more than 220 million order insertions, which approaches that bound. The head-to-head

benchmark therefore runs on the desktop (32 GB), and we have to confirm CppTrader's array is large enough for the 2020 reference numbers, or the comparison would be measuring a crash.

Repositories and delivery

Development happens in a private repository with full history (`itchbook-dev` on GitHub, `~/CODE/itchbook` locally). A public copy (`itchbook`, `~/CODE/itchbook-public`) receives a single clean commit per release from `scripts/sync_public.sh`. That script exports exactly what is in the current commit, so nothing untracked can leak into the public copy. Work is delivered to the Mac as git bundles, which are complete repositories in a single file, placed in the repository's ignored `target/` directory and pulled from there. Every bundle is built on top of the commit the Mac already has, and each one is checked on the Mac after transfer.

Questions to expect

Why not just parse into structs? Copying every message into a struct costs a write of 20–50 bytes per message, 400 million times. The views decode only the fields that are actually read, straight from the input buffer. The M1 walkthrough covers this in detail.

Why both property tests and fuzzing? Property tests check that valid input is handled correctly, with exact expected output. The fuzzer checks that arbitrary input, valid or not, can't corrupt memory or break accounting. Neither covers the other.

How do you know the sanitizer build tests the same code? It builds the same sources with instrumentation added, and the same test suite runs. Performance numbers never come from it.

What would make the CppTrader comparison unfair, and how do you avoid it? Different input files, different machines, compressed input on one side and uncompressed on the other, or timing that includes different work. The comparison uses the same uncompressed file on the same machine, single-threaded, and reports per-message latency percentiles, not just an average.